VISUAL NOTES • COMPLETE GUIDE

Visual Notes — Self-Improving Agents and Skills

Self-Improving Agents and Skills

18:31

- 23 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Claims, uncertainty, and caveats are labeled clearly

https://www.youtube.com/watch?v=u-o0sW9nwmk

Turn Every Hard-Won Solution into a Playbook

A solved problem becomes reusable guidance instead of disappearing with the session.



NOTES

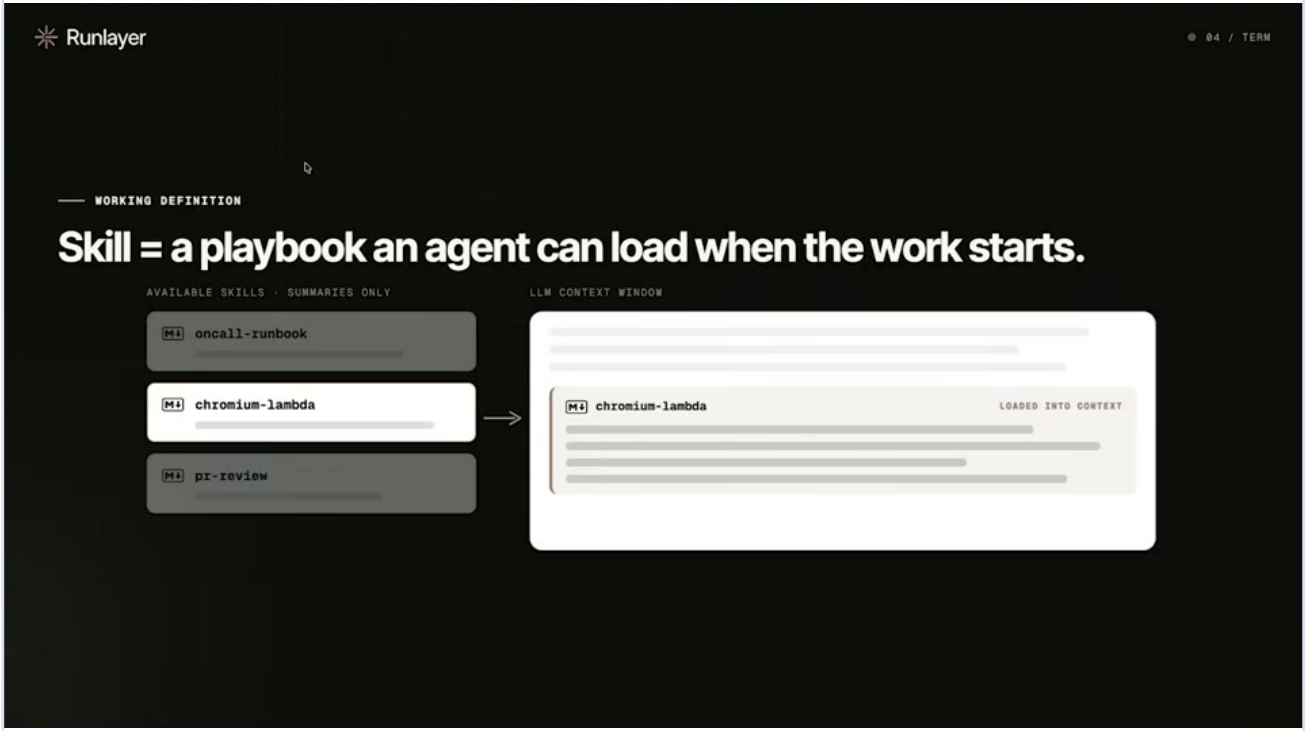
- A playbook records the steps, decisions, and context that made a difficult solution work.
- Without capture, debugging paths and reasoning vanish when a task or session ends.
- Automatic capture would let later agents reuse a solution rather than reconstruct it from scratch.
- The organizational goal is to make learning from one difficult task available across many future tasks.

CORE IDEA

Durable playbooks convert isolated problem-solving into reusable organizational knowledge.

Skills Guide an Agent Only When Relevant

Progressive disclosure keeps guidance available without loading every procedure up front.



NOTES

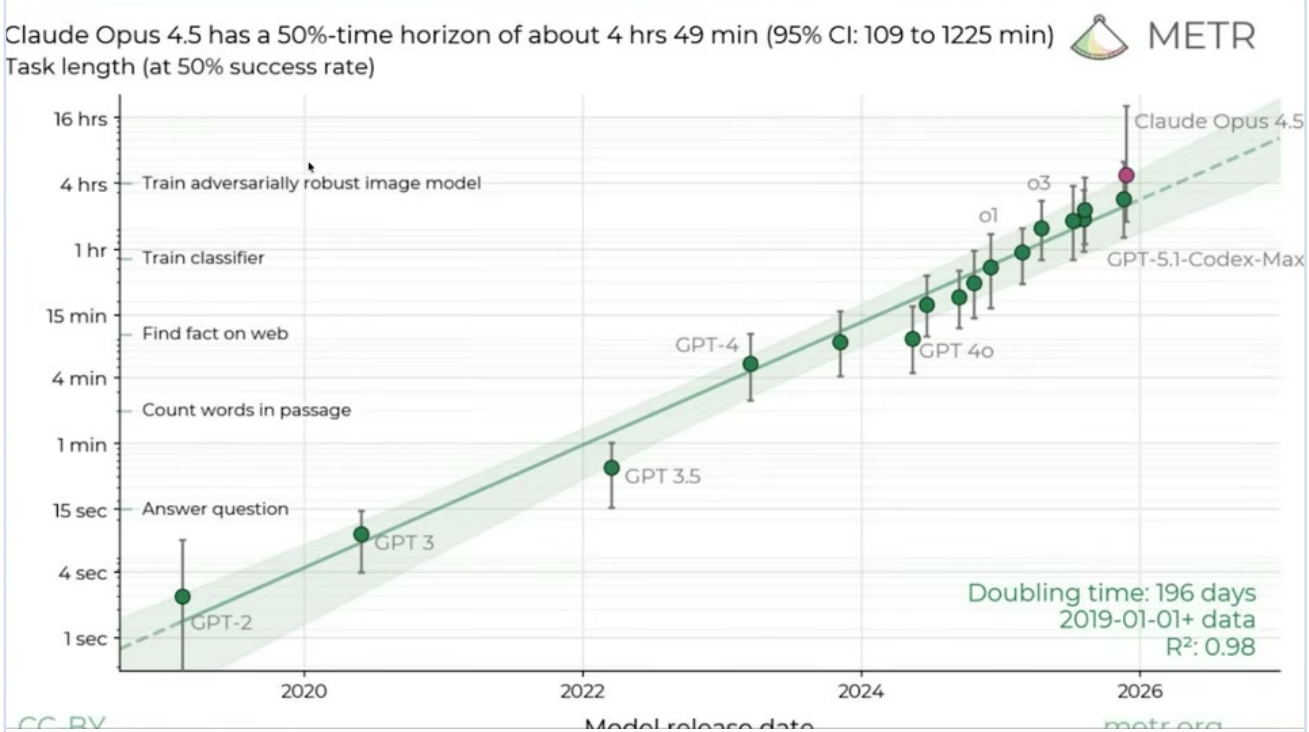
- A skill is a reusable playbook an agent can read when its knowledge is relevant to the current task.
- Progressive disclosure means showing a compact skill summary first, then loading the full instructions only when needed.
- Once loaded into context, the skill can redirect the agent toward a proven sequence of actions.
- Selective loading preserves context space while keeping specialized procedures discoverable.

CORE IDEA

Skills make procedural knowledge available at the decision point without crowding every task with every instruction.

Longer Autonomous Work Expands the Stakes

A cited task-horizon chart connects model progress with longer independent work.



NOTES

- The chart presents a METR-style task horizon: how long an agent can work independently while retaining a stated chance of success.
- Its upward trend is used to argue that frontier systems can attempt increasingly long, multi-step tasks.
- A probability of success is not a guarantee; longer task duration still leaves substantial room for failure.
- The displayed trend is a cited performance claim, not an independently reproduced benchmark in these notes.

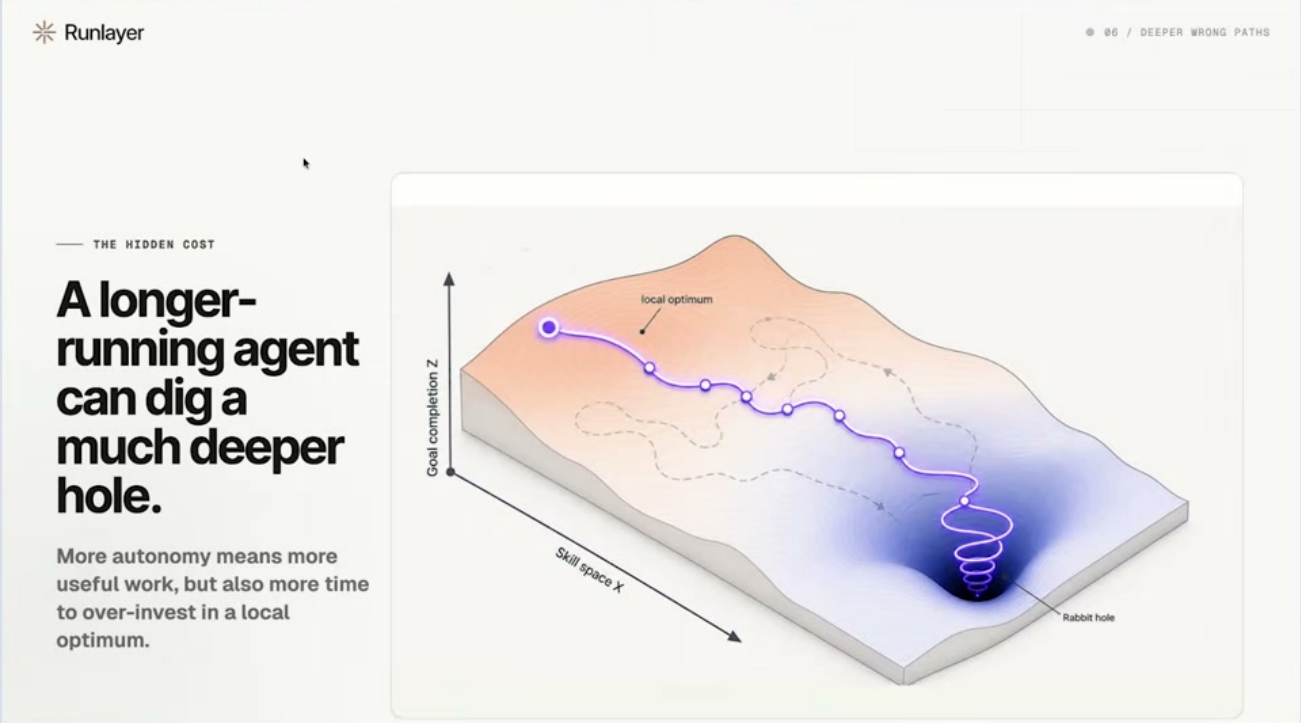
CORE IDEA

As autonomous task horizons grow, early guidance has more time to amplify either progress or error.

Source: https://www.youtube.com/watch?v=u-o0sW9nwmk

A Bad Initial Trajectory Becomes a Deep Rabbit Hole

Long reasoning runs can spend many tool calls defending the wrong premise.



NOTES

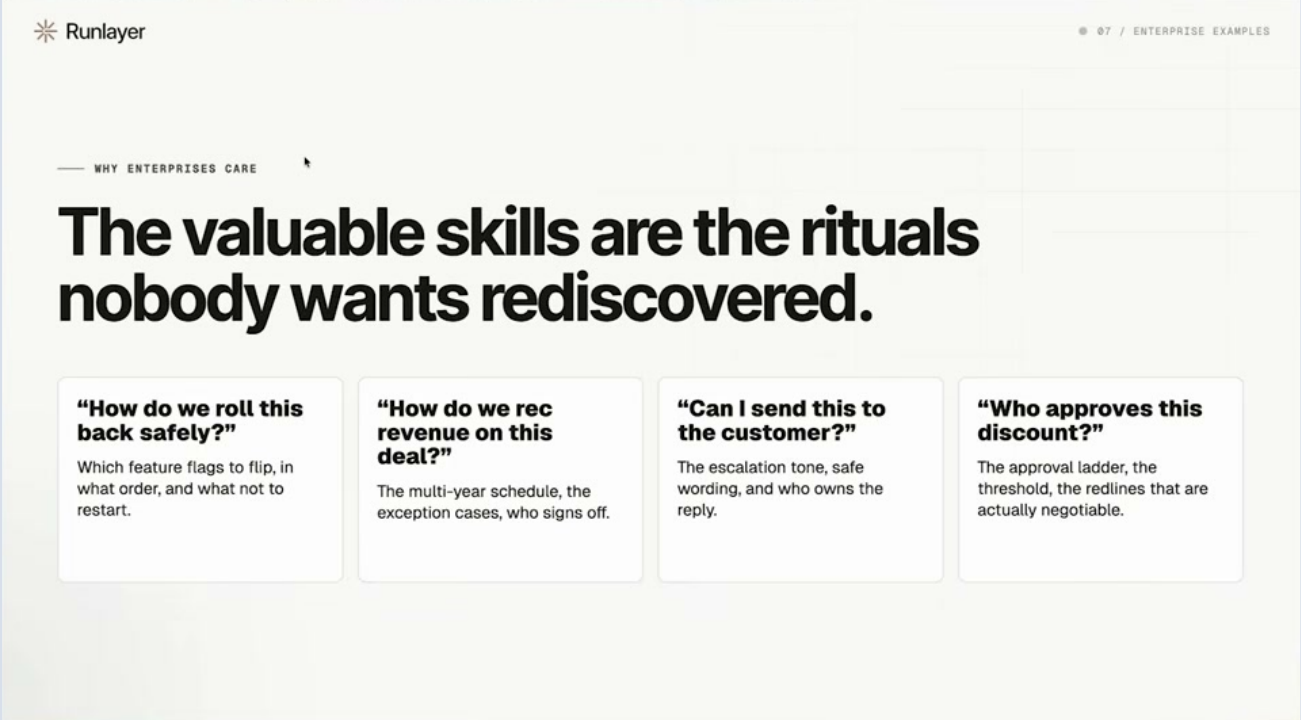
- An agent that starts from a false thesis may continue exploring and defending it for hours.
- Hundreds of tool calls can accumulate before the run reveals that the initial direction was wrong.
- A later agent can repeat the same mistake if the failed path is not captured and shared.
- Skills can constrain the initial search toward known-good approaches, reducing wasted time, tokens, and money.

CORE IDEA

Long-task capability increases the value of steering early, before an error compounds through an expensive run.

Capture the Internal Rituals Agents Cannot Guess

The most valuable procedures are often specific, mundane, and costly to rediscover.



NOTES

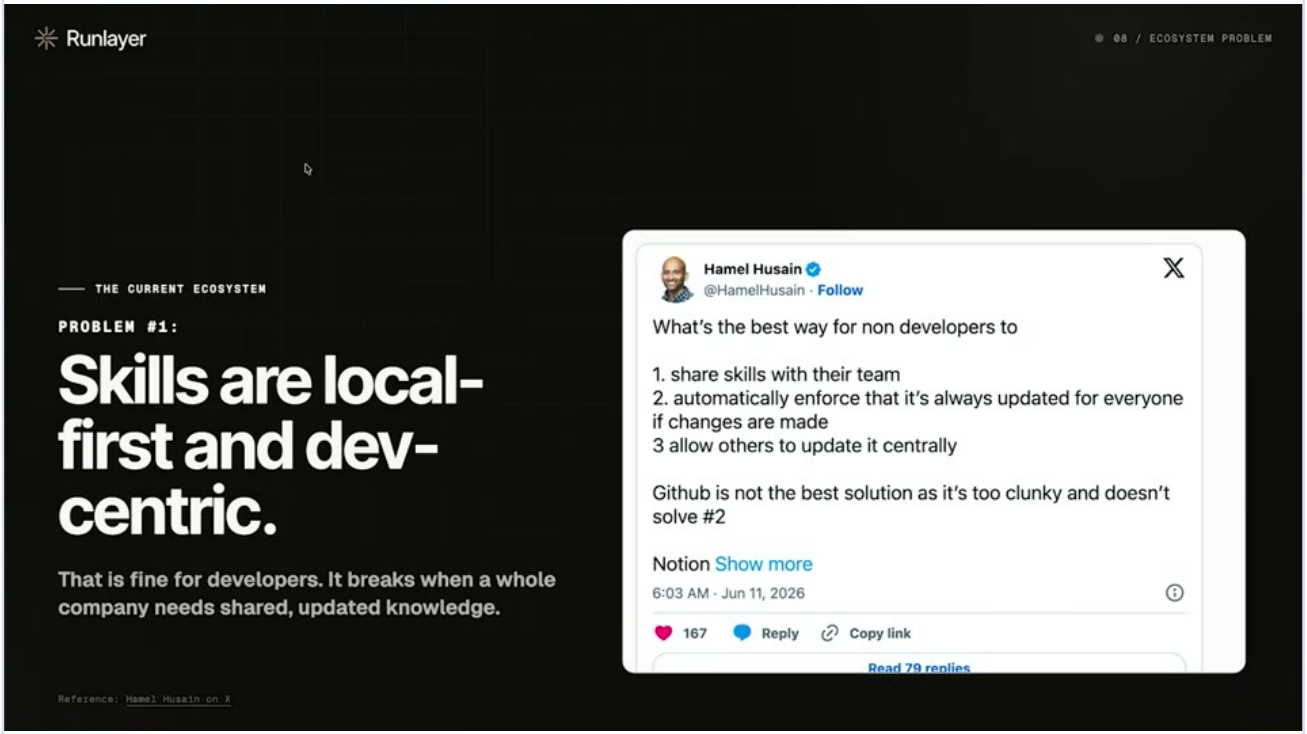
- High-value knowledge often lives in internal sequences rather than public documentation.
- Examples include the exact scripts and feature flags for a safe rollback or approved language for a sensitive customer interaction.
- These rituals encode local systems, policies, and experience that a general model does not possess.
- Expecting an agent to rediscover them during a live task creates avoidable operational risk.

CORE IDEA

Local operating knowledge is where reusable skills can prevent the costliest kinds of improvisation.

Three Problems Block Skill Adoption

Creation, distribution, and client compatibility prevent useful procedures from scaling.



NOTES

- First, the ecosystem is local-first and developer-centric, relying on files, command lines, and repositories.
- Second, writing a durable playbook requires reflection and effort that busy teams rarely prioritize.
- Third, skill support differs across agent clients, especially outside technical teams.
- Solving only the file format leaves the human adoption and distribution problems intact.

CORE IDEA

Skills become organizational infrastructure only when people can create, trust, find, and use them across clients.

Source: https://www.youtube.com/watch?v=u-o0sW9nwmk

Local Skill Installation Creates a Trust Boundary

Convenient packages may combine instructions, scripts, permissions, and secrets from unknown sources.



NOTES

- A downloaded skill can contain Markdown instructions, executable scripts, dependency commands, or requests for elevated permissions.
- Unreviewed content can introduce prompt injection, unsafe commands, or secret-handling mistakes.
- The visual's security metaphor is illustrative rather than a complete threat model or measured prevalence claim.
- Practical controls include provenance, review, sandboxing, least privilege, and scanning before organization-wide use.

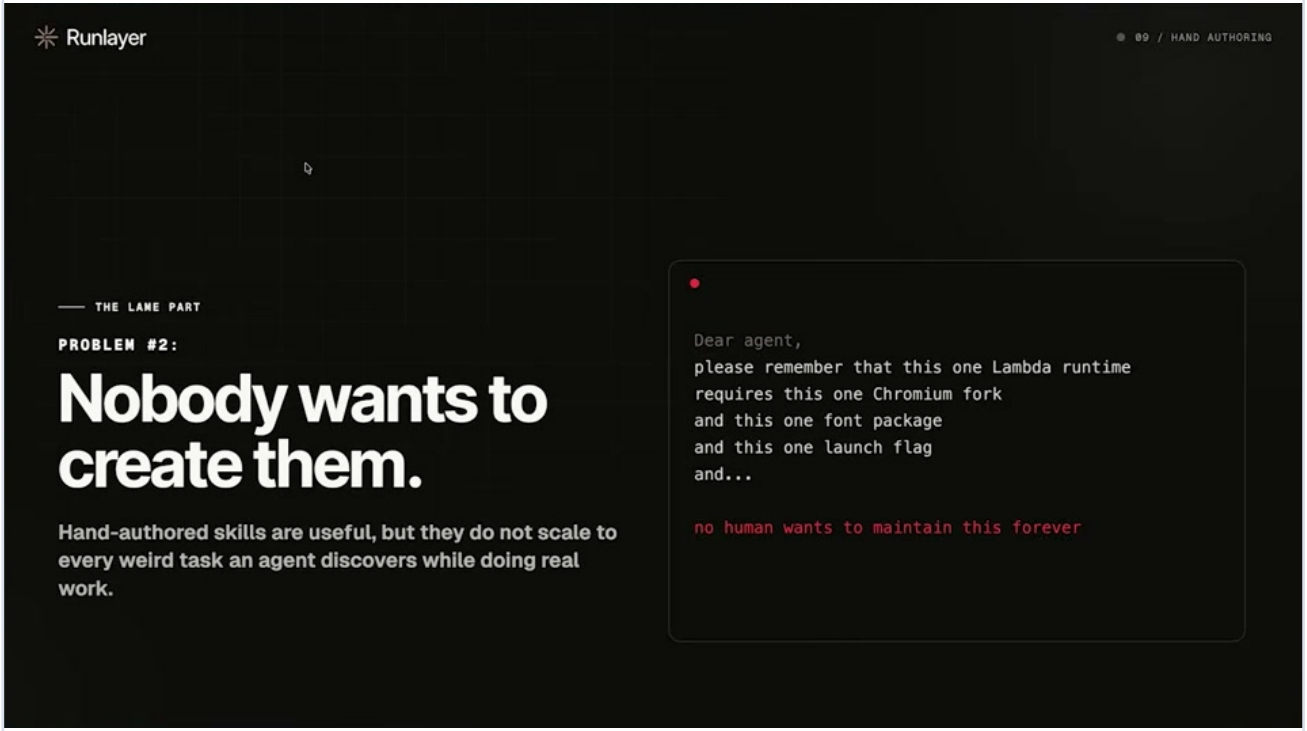
CORE IDEA

Skill reuse must preserve the trust controls expected of any code and instruction supply chain.

Source: https://www.youtube.com/watch?v=u-o0sW9nwmk

Useful Playbooks Usually Do Not Exist Yet

Reflection after hard work competes with the next urgent task.



NOTES

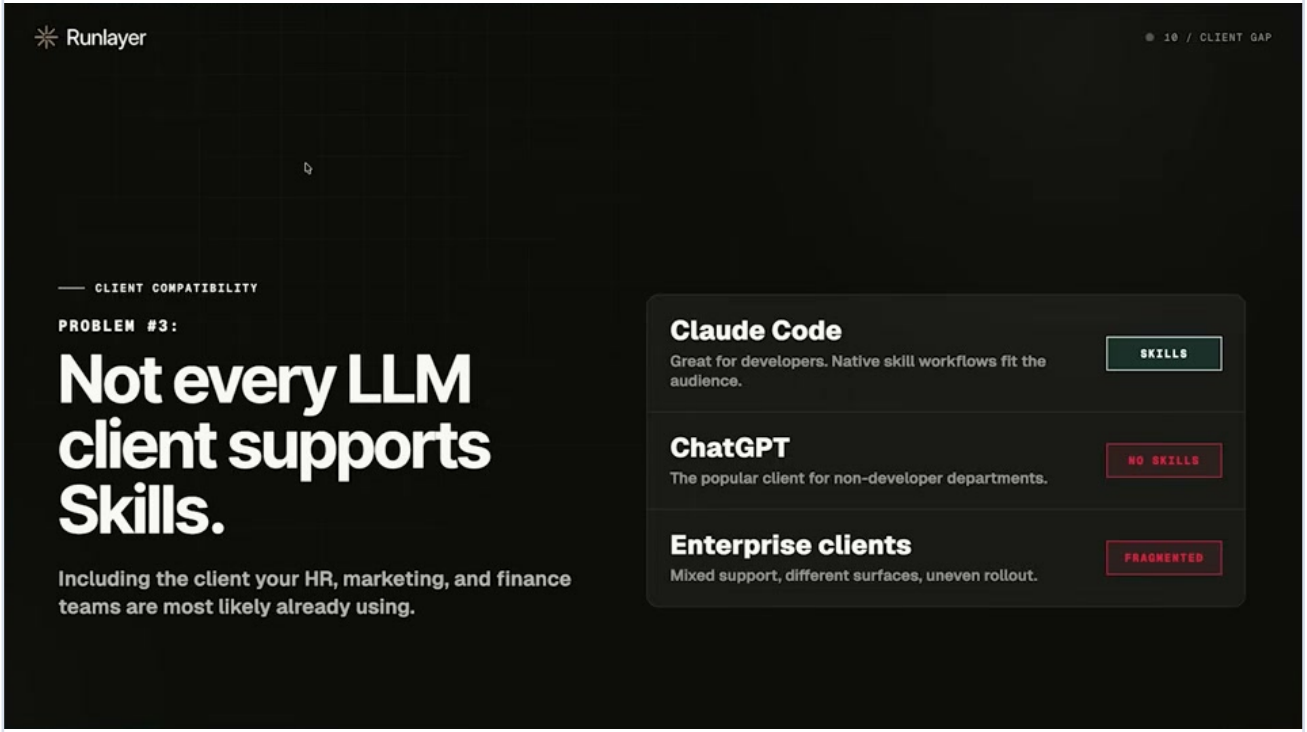
- Teams value a ready-made procedure when a difficult situation appears, but many situations have no playbook.
- Producing one requires revisiting the run, separating signal from noise, and writing steps future users can apply.
- That work demands time, intention, and maintenance, so it is often postponed.
- Automated extraction from completed work is proposed as a way to lower this creation burden.

CORE IDEA

The skill library cannot compound if capturing knowledge remains an optional documentation chore.

Client Fragmentation Limits Who Can Use Skills

Technical and nontechnical teams often work through different agent interfaces.



NOTES

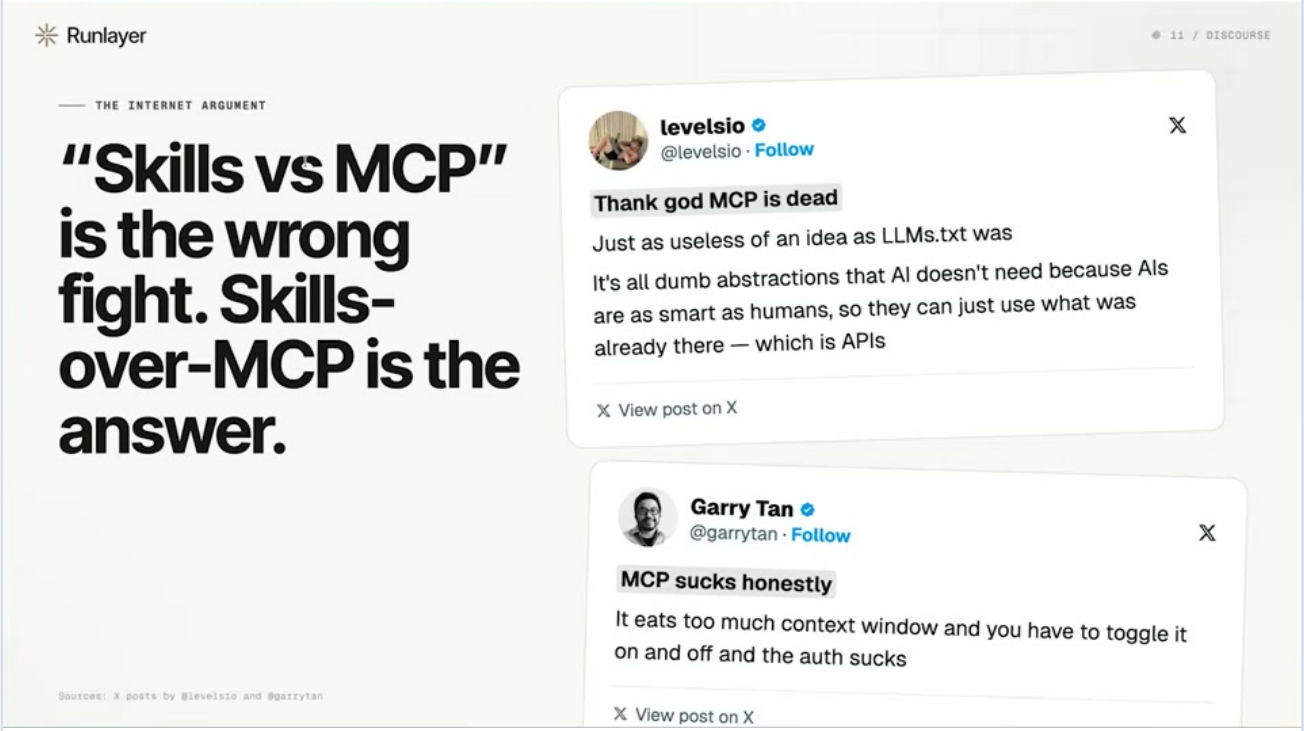
- Native skill support is inconsistent across agent clients and may require client-specific setup.
- Developers can often work around gaps with files and command-line tools, but that path does not scale to every department.
- Marketing, HR, legal, and finance may use interfaces that cannot install or discover local skill packages.
- A common delivery mechanism is needed if one procedure should reach the entire organization.

CORE IDEA

Knowledge sharing fails when the delivery path works only for the teams most comfortable with developer tooling.

Skills and MCP Solve Different Problems

Procedural knowledge and executable capabilities are complementary layers.



NOTES

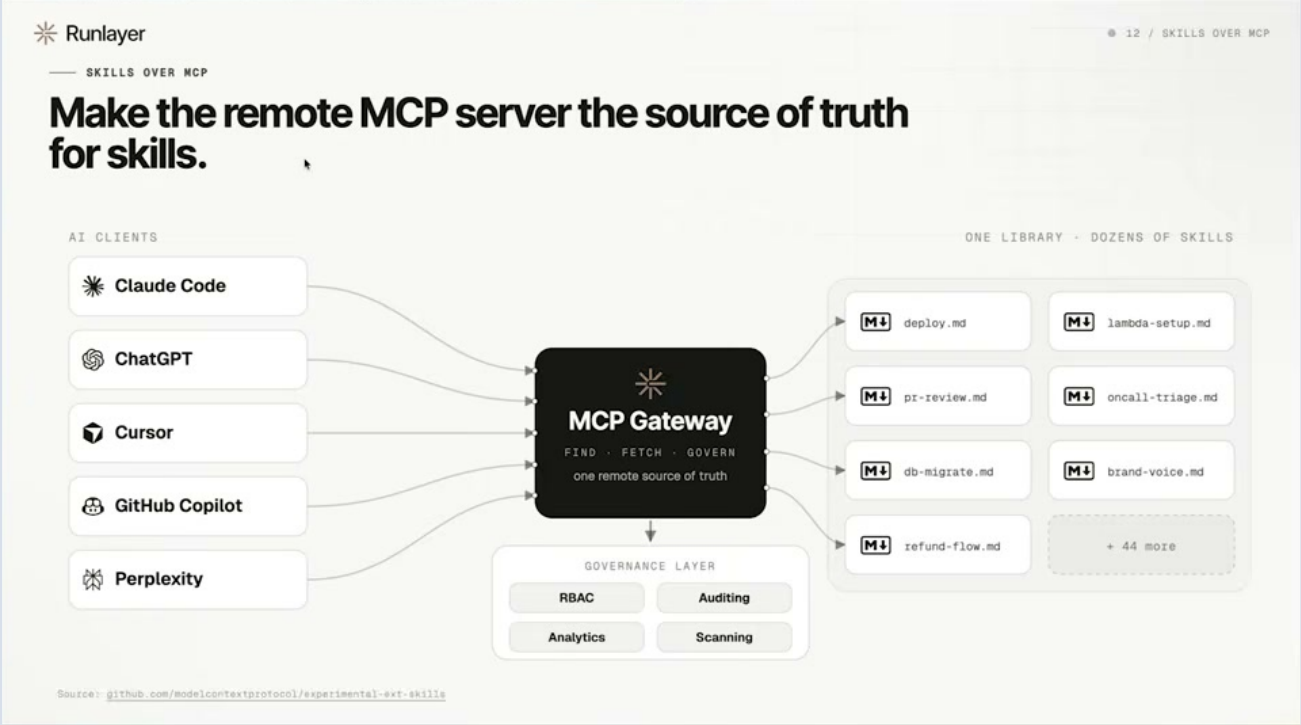
- A skill supplies knowledge about how to perform a task: sequence, constraints, checks, and local conventions.
- MCP, the Model Context Protocol, is a standard interface through which an agent client can discover and invoke remote tools or resources.
- Tools provide capabilities to act; skills provide procedures for selecting and combining those capabilities.
- MCP can also carry skill content, making it a distribution layer rather than a replacement for skills.

CORE IDEA

Combining procedural guidance with interoperable tools gives agents both a method and the means to execute it.

A Remote MCP Server Becomes the Source of Truth

Clients search one governed service instead of installing independent local copies.



NOTES

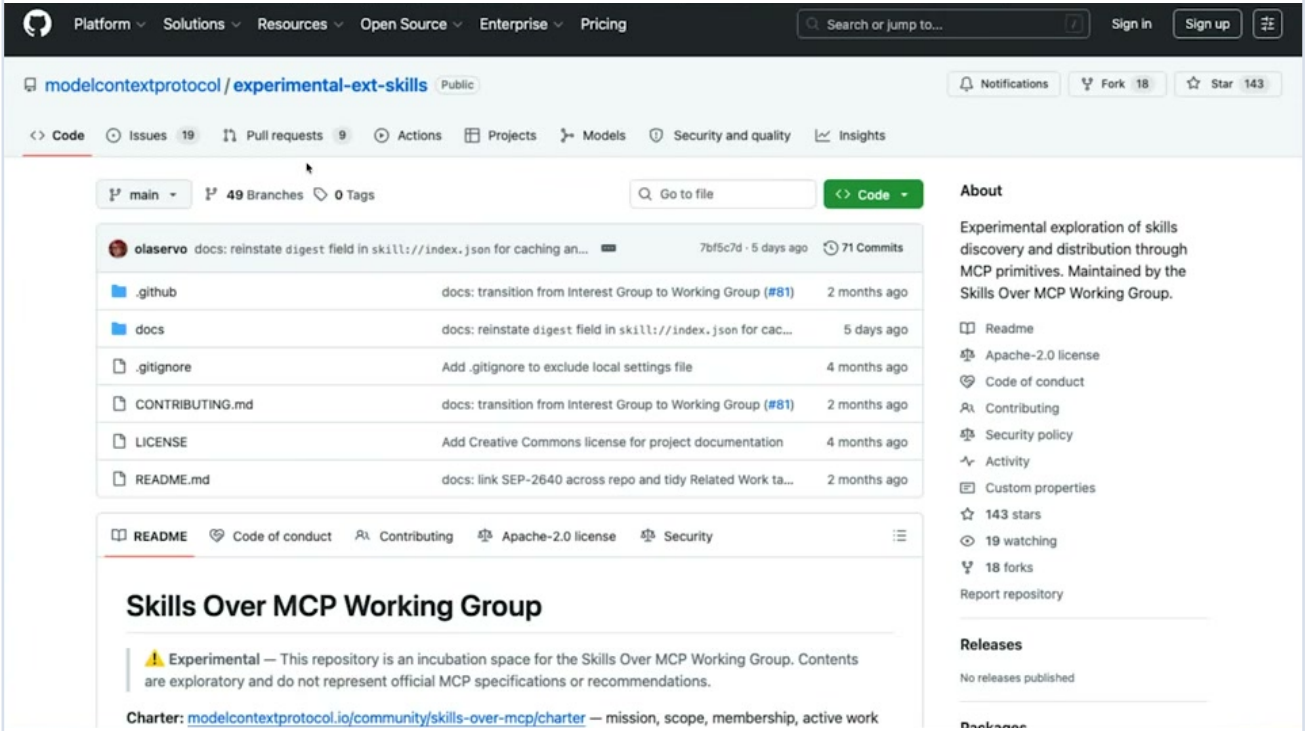
- A remote MCP service can expose a searchable catalog of approved skills to many clients.
- Central storage avoids divergent local copies and can return the best available version for a query.
- RBAC, or role-based access control, grants knowledge according to a user's assigned role rather than exposing every skill to everyone.
- Policy checks, versioning, and access decisions can therefore occur at one managed boundary.

CORE IDEA

Central delivery turns scattered files into a governable, versioned knowledge service.

Skills over MCP Is an Experimental Direction

A working group explores shared delivery patterns, but the exact interface is not final.



NOTES

- The referenced Skills over MCP working-group page explores how MCP could distribute reusable knowledge.
- It should be treated as experimental community work, not as an official finalized MCP specification.
- The direction supports centralized discovery while preserving client interoperability.
- Implementations should expect the API shape and recommended transport to change.

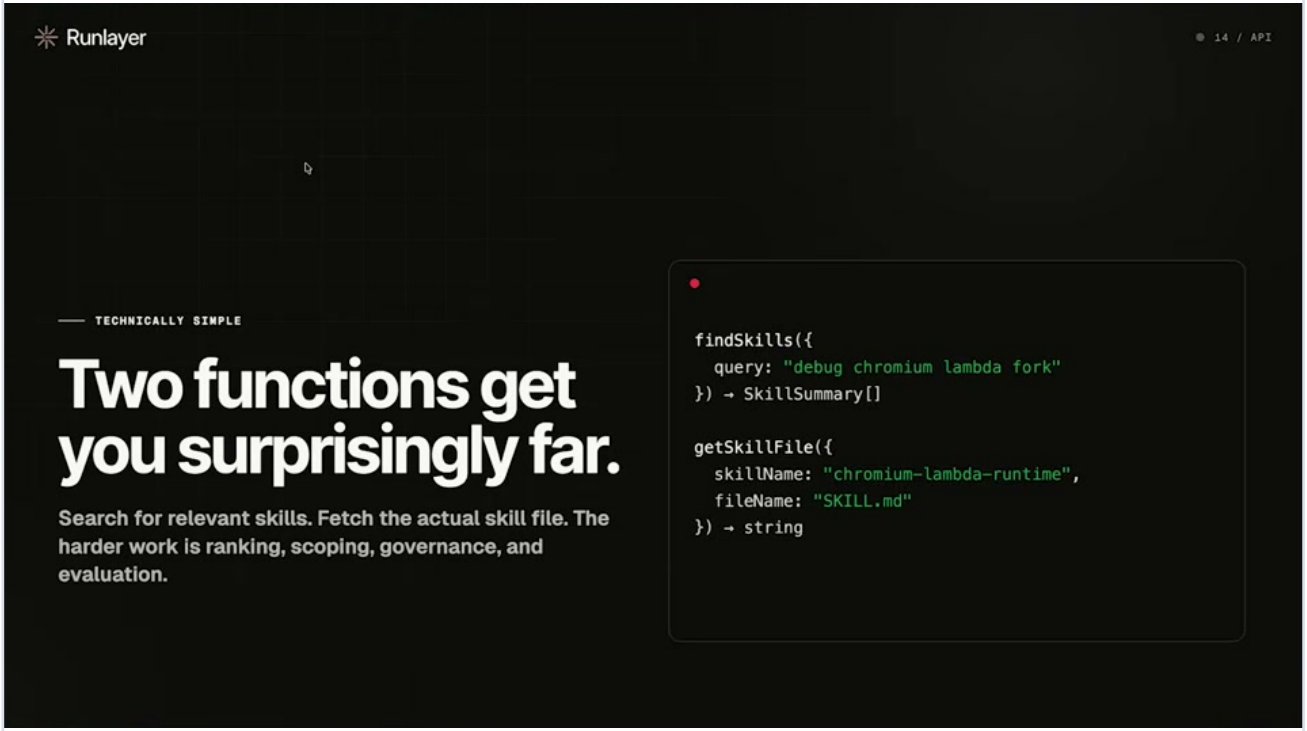
CORE IDEA

The distribution idea is actionable, but production designs should not mistake an experiment for a stable standard.

Source: https://www.youtube.com/watch?v=u-o0sW9nwmk

Resources or Tools: the API Is Still Settling

Mixed client support can force a pragmatic implementation choice.



NOTES

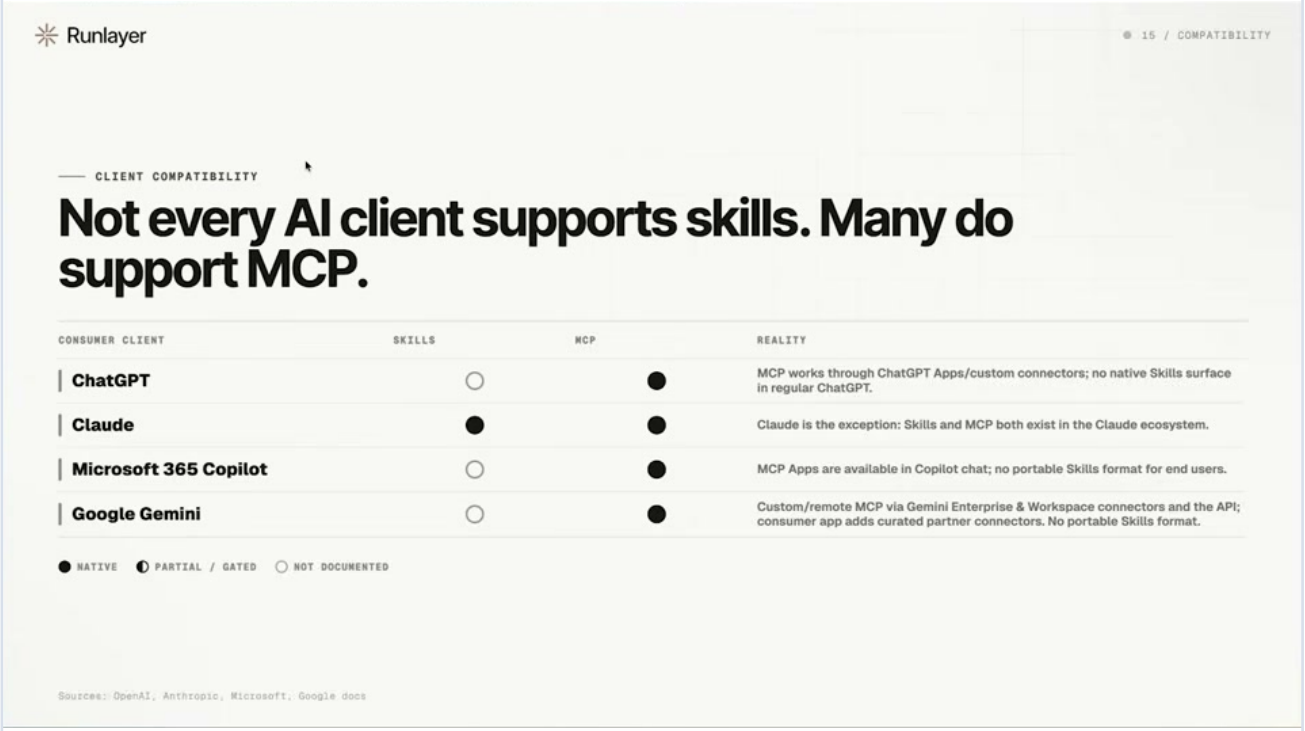
- MCP resources are a natural representation for retrievable knowledge, but resource support varies by client.
- Tool calls offer a more widely supported way to search for and fetch skills today.
- The proposed API is implementation-specific and experimental rather than a normative protocol requirement.
- A migration path is prudent so delivery can evolve as client and protocol support matures.

CORE IDEA

Interoperability may require a temporary tool-based bridge while the shared API and client ecosystem mature.

Govern Distribution from One Control Plane

Central policy connects reusable knowledge to technical and nontechnical clients.



NOTES

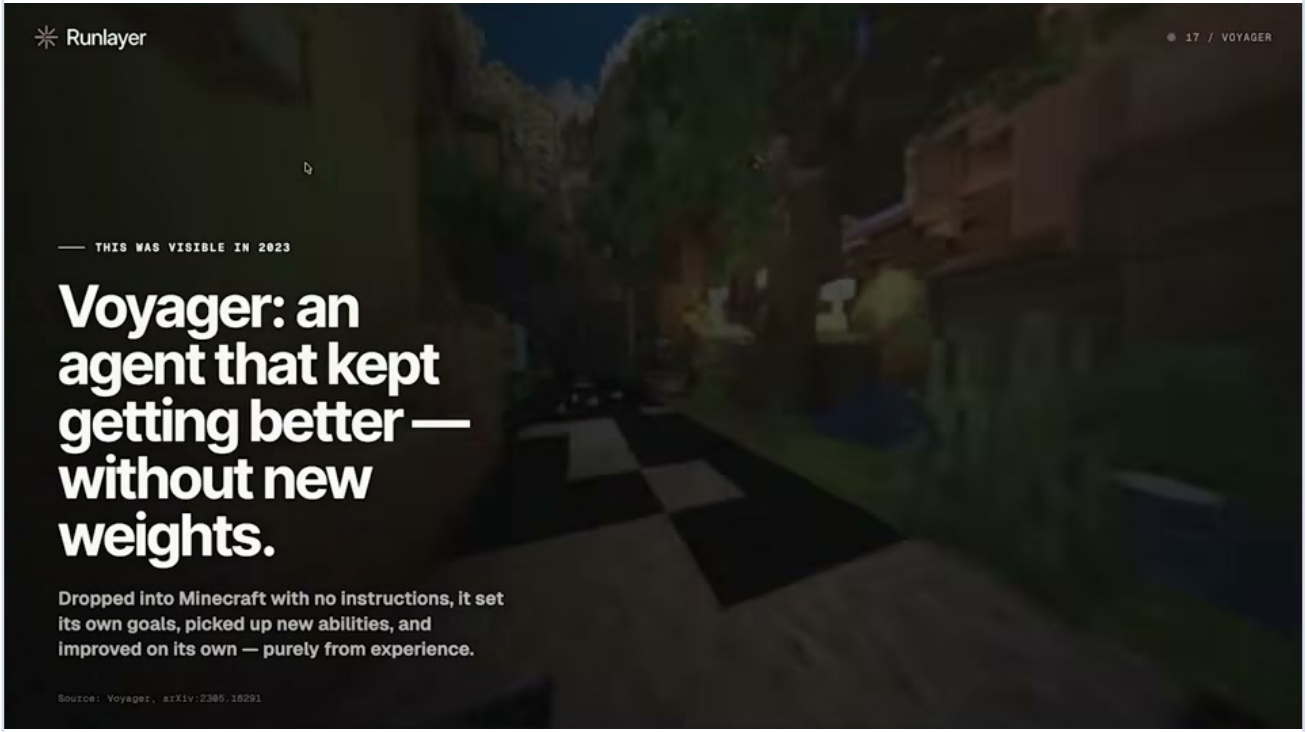
- A centralized service can manage which skills exist, which versions are current, and who may access them.
- The same interface can serve multiple agent clients instead of requiring separate local installation paths.
- Governance can include approval status, ownership, audit history, sensitive-data checks, and rollback.
- Broad client compatibility addresses distribution, while policy enforcement addresses organizational control.

CORE IDEA

A shared control plane makes procedural knowledge broadly reachable without giving up ownership and policy enforcement.

Voyager Shows Learning Without Changing Model Weights

An autonomous Minecraft agent improves by accumulating reusable procedures.



NOTES

- Voyager is a 2023 research system in which an LLM-driven agent explores Minecraft, proposes goals, and builds capabilities over time.
- The agent begins without a fixed task script and learns through interaction with the environment.
- Improvement comes from memory and reusable code rather than retraining or modifying the underlying model weights.
- The example supplies a conceptual precedent for agents that retain what successful exploration discovers.

CORE IDEA

Persistent procedural memory can improve future behavior even when the underlying model remains unchanged.

The Skill Library Preserves Successful Recipes

Useful action sequences are stored as executable procedures for later retrieval.

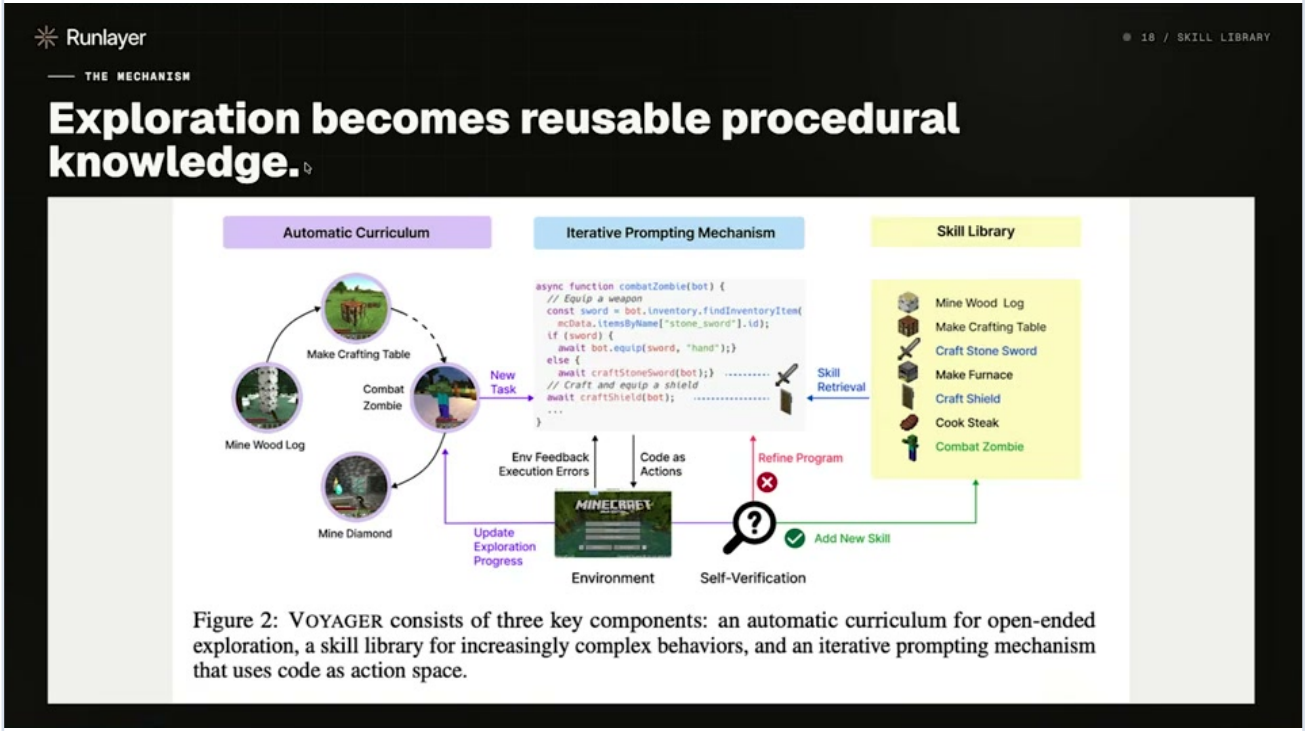


Figure 2: VOYAGER consists of three key components: an automatic curriculum for open-ended exploration, a skill library for increasingly complex behaviors, and an iterative prompting mechanism that uses code as action space.

NOTES

- When exploration discovers how to make an item or complete a subtask, Voyager stores the solution as code.
- The stored recipe represents the tool calls and control logic that led to the result.
- Later tasks can retrieve that recipe instead of exploring the same branches again.
- A growing library lets simple capabilities compose into more complex behavior.

CORE IDEA

Saving successful procedures turns one-time exploration into a cumulative capability library.

Convert Exploration into Procedural Knowledge

The transferable pattern is to keep the method, not merely the outcome.



NOTES

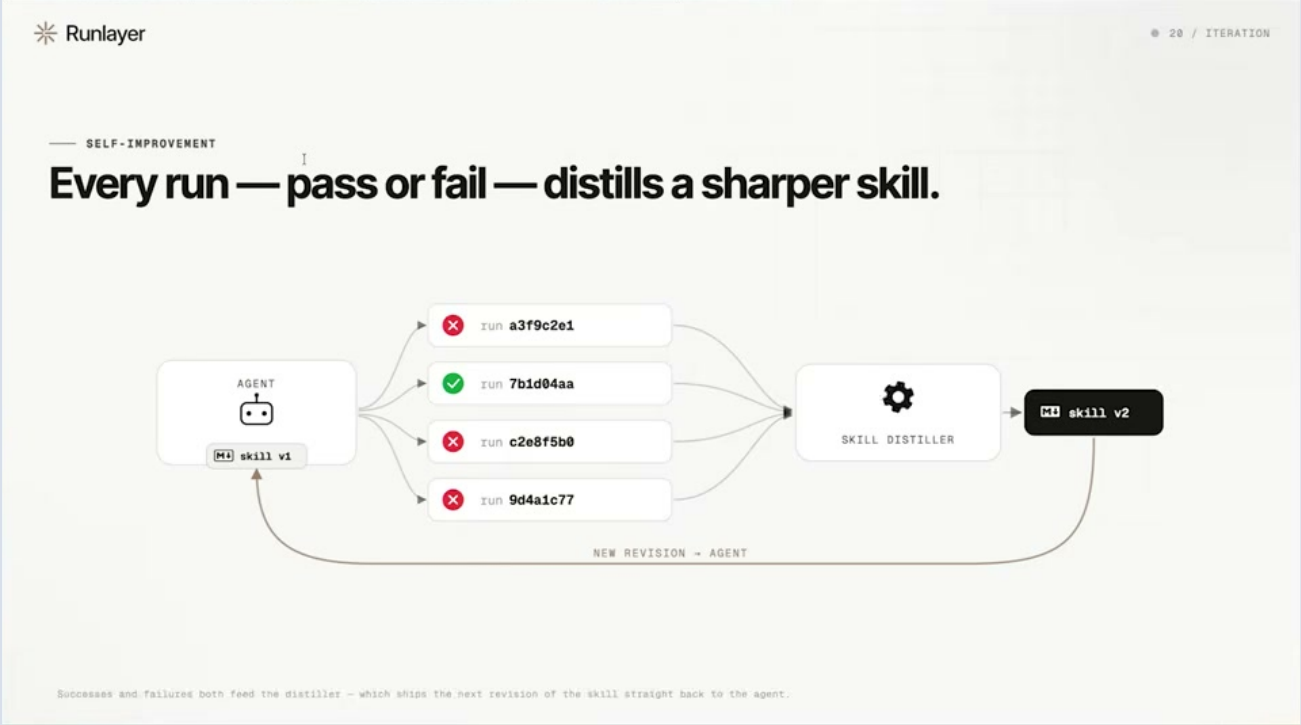
- A completed task contains more value than its final answer: it also contains a path through decisions and tool use.
- Throwing away that path forces future agents to pay the exploration cost again.
- Reusable procedures should capture prerequisites, action order, checks, and recovery steps rather than raw logs alone.
- The procedure must remain narrow enough to retrieve for the right future situation.

CORE IDEA

The durable asset is a reusable method that changes future search, not a static record of one result.

Distill Skills from Real Agent Traces

Completed work is compressed into a reusable procedure.



NOTES

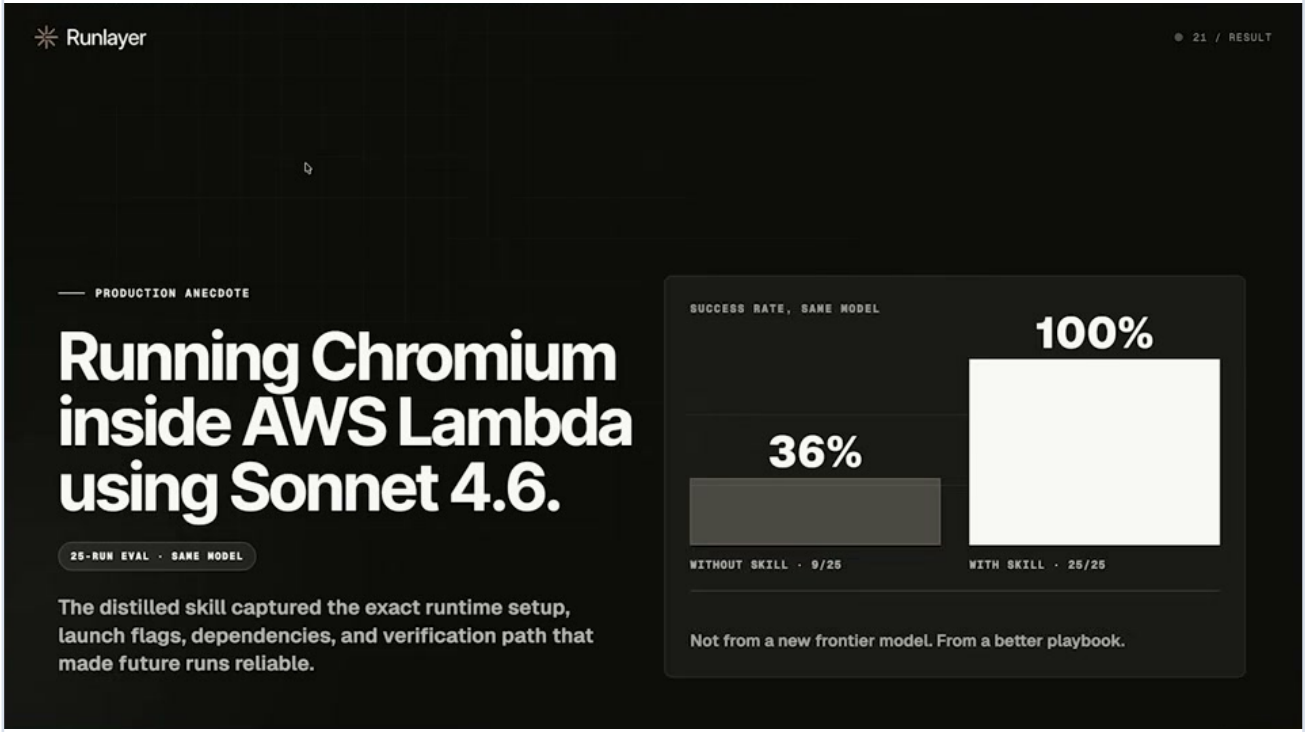
- A trace is the recorded sequence of an agent's messages, decisions, tool calls, observations, and outcomes during a run.
- Distillation is the process of compressing that detailed trace into concise, reusable guidance while removing incidental noise.
- A capable model can inspect a successful multi-step run and draft a skill containing the repeatable procedure.
- Generated skills still need checks for correctness, scope, sensitive data, unsafe instructions, and maintainability.

CORE IDEA

Trace distillation lowers the cost of turning actual work into reusable operational knowledge.

Failed Runs Expose Missing Guardrails

Errors reveal edge cases and safeguards that a success-only library would miss.



NOTES

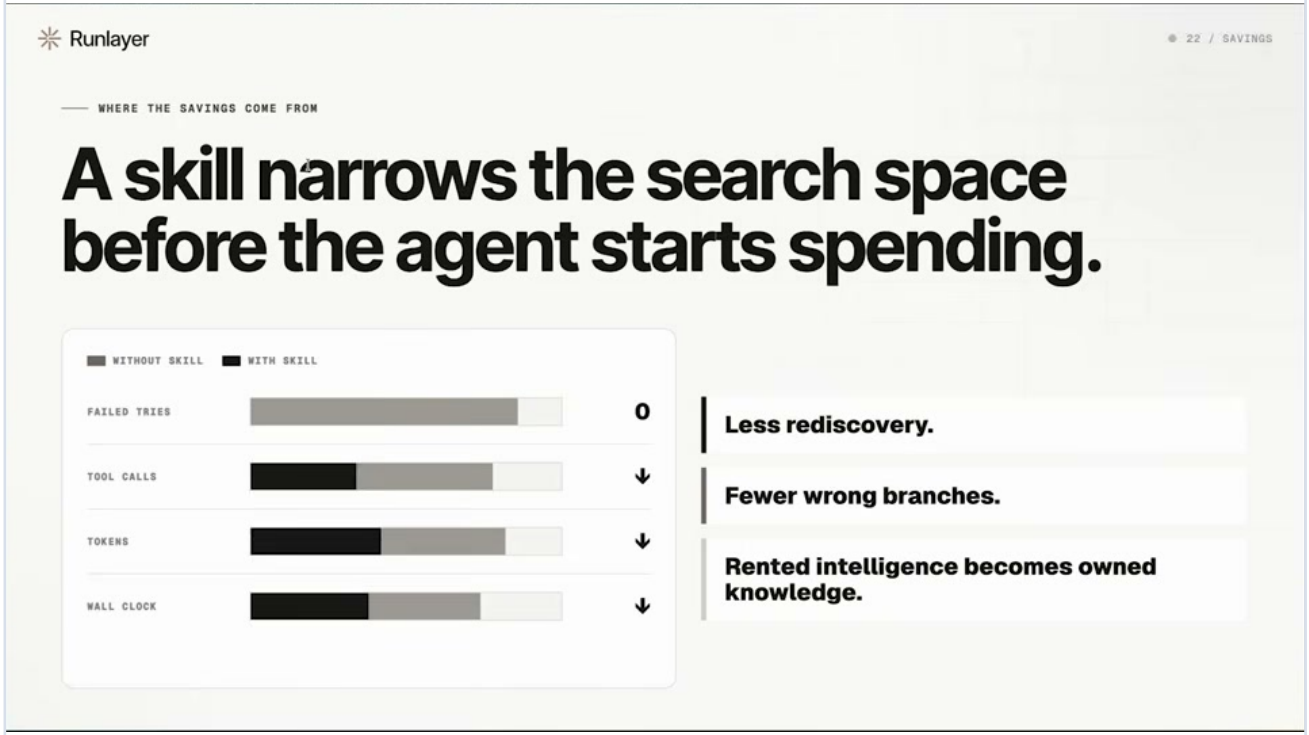
- A failed run can identify the exact branch where assumptions, dependencies, or checks broke down.
- Failures reveal missing guardrails, undocumented libraries, environmental constraints, and unanticipated edge cases.
- Successful runs increase confidence in a procedure; failed runs supply corrections that can make it more robust.
- Automated updates require evaluation and version control so a noisy failure does not degrade a good skill.

CORE IDEA

Learning from both outcomes produces procedures that cover recovery and prevention, not only the happy path.

A Demo Reports a Shift from 36% to 100%

One solved run supplies a skill reused by later attempts.



NOTES

- The demonstrated task involved running a Chromium fork in AWS Lambda with a specified model configuration.
- Before skill reuse, the reported success rate was 36% across the demonstration's evaluation setup.
- After one run succeeded, a distilled skill was inserted into later attempts, which were reported at 100% success.
- The 36%→100% result is a source claim from a specific demo, not an independent benchmark or guarantee of generalization.

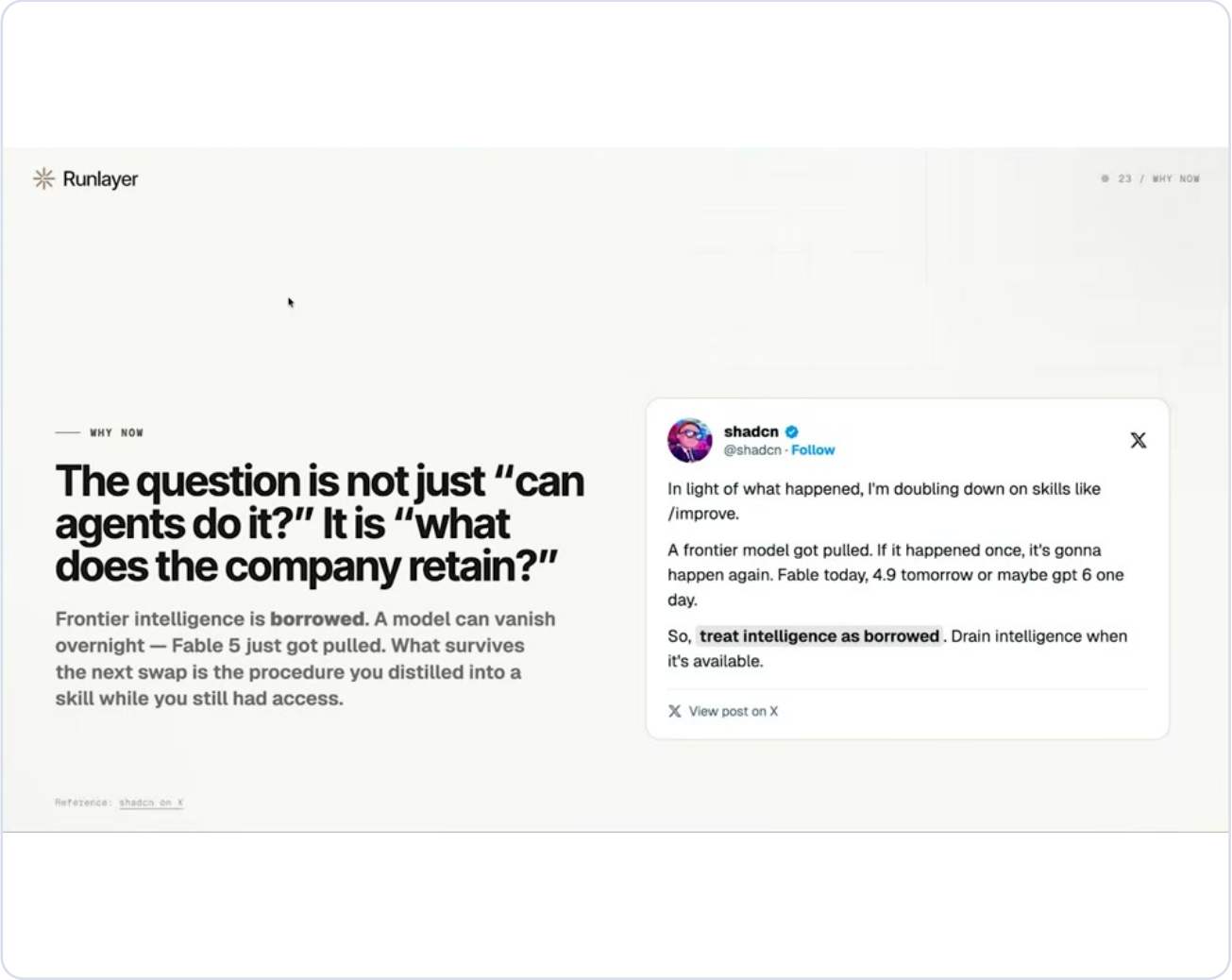
CORE IDEA

A single successful path may sharply improve repeated tasks, but the effect must be validated beyond the demo setting.

Source: https://www.youtube.com/watch?v=u-o0sW9nwmk

Good Skills Narrow the Search Space

The payoff includes fewer branches, tool calls, tokens, and retries.



NOTES

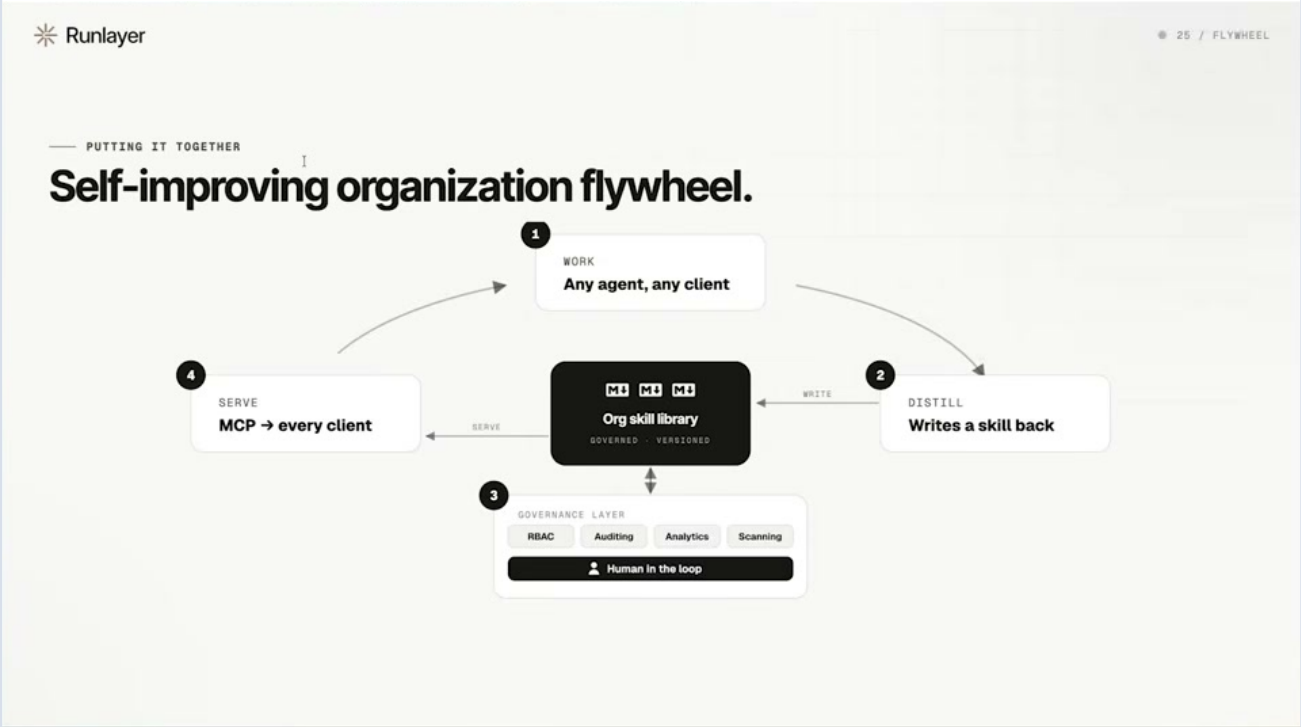
- Success rate is only one outcome; a useful skill also removes branches already known to be unproductive.
- A narrower search can reduce exploratory tool calls and shorten the path to a valid result.
- Fewer calls and less context consumption can lower latency, token use, and execution cost without changing models.
- These efficiency effects are proposed consequences and should be measured on the target workload.

CORE IDEA

Procedural knowledge can improve economics by reducing wasted search, not only by increasing task completion.

Frontier Intelligence Is Rented

Model capability can change, while captured procedures remain under organizational control.



NOTES

- Access to a frontier model is borrowed from an external provider rather than owned as a durable internal asset.
- Models can be deprecated, repriced, replaced, or behave differently after updates.
- Strong models can be used while available to discover procedures and record how work succeeds in the local environment.
- Persisted skills can then retain part of that practical value across model changes, although they still require maintenance.

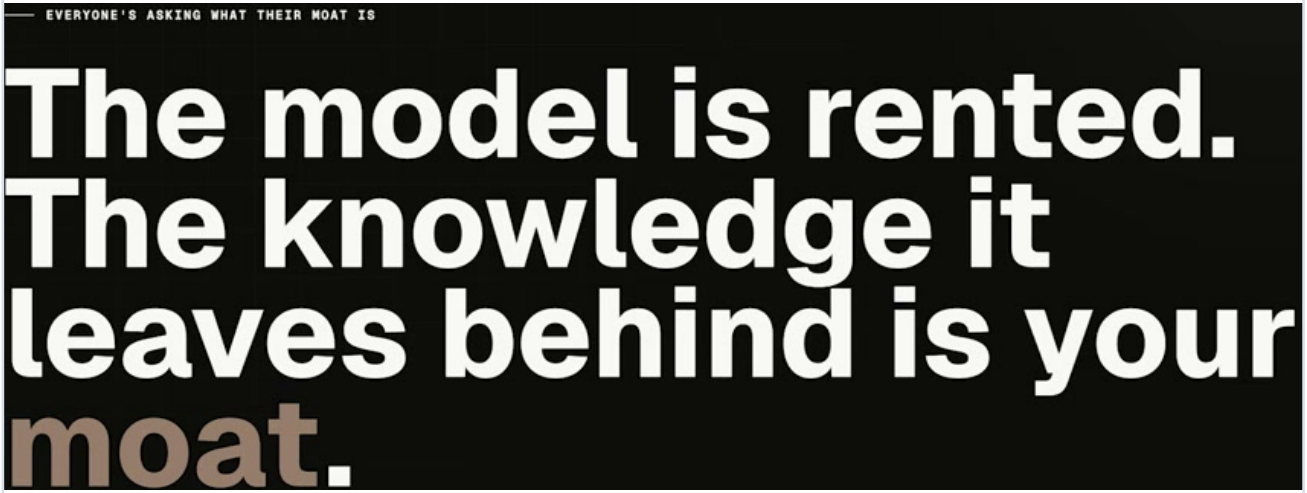
CORE IDEA

Capturing procedures converts temporary access to model capability into knowledge the organization can retain.

Source: https://www.youtube.com/watch?v=u-o0sW9nwmk

The Organizational Flywheel Becomes the Moat

Runs produce governed skills that improve every connected agent and generate better future runs.



NOTES

- An organizational flywheel is a reinforcing loop in which many runs generate lessons, those lessons become governed skills, and shared skills improve later runs.
- Candidate skills are grouped and checked for hazards such as prompt injection and personally identifiable information before entering the library.
- MCP distribution makes an approved lesson available across connected clients, so one agent's discovery can raise the baseline for others.
- Models are widely available, but accumulated local procedures, edge cases, culture, and operating history are difficult for competitors to copy.
- The resulting knowledge moat compounds only if evaluation, ownership, access control, and updates remain continuous.

CORE IDEA

A governed learning loop can turn private operating experience into a compounding knowledge asset that survives model churn.